Arturito — esquema de conexión y código

Robot seguidor de línea · Gaby Leryn Ayala Chambi, Christopher Darius Valero Lima y Maximiliano Sosa Useche · U.E. Jaime Alfonso Escalante Gutierrez A, Santa Cruz de la Sierra · 2026

Esquema de conexión

Sensores → Arduino UNO

Los cinco sensores TCRT5000 entran por los pines analógicos. Mirando al robot de frente, S0 queda a la izquierda y S4 a la derecha.

Señal	Pin
S0 (extremo izquierdo)	A0
S1 (centro izquierdo)	A1
S2 (centro)	A2
S3 (centro derecho)	A3
S4 (extremo derecho)	A4

UNO → DRV8833 (driver de motores)

Señal	Pin

IN1 (motor izquierdo, adelante)	D5
IN2 (motor izquierdo, atrás)	D6
IN3 (motor derecho, adelante)	D9
IN4 (motor derecho, atrás)	D10
EEP (nSLEEP; HIGH = driver despierto)	D4
ULT (nFAULT, opcional)	D2

Sin el cable a EEP el chip queda dormido: enciende su LED pero las salidas no conducen.

DRV8833 → motores

- OUT1/OUT2 'motor izquierdo
- OUT3/OUT4 'motor derecho

Si un motor gira al revés, se intercambian sus dos cables en OUT o sus dos entradas IN; da igual por dónde.

Alimentación

Cuatro pilas AA alimentan el conjunto; la masa es común entre sensores, driver y UNO.

Código — versión 1 (`firmware/arturito/arturito.ino`)

Cada lectura se convierte a blanco o negro con un umbral (500) y el movimiento se decide con una tabla de casos. Al encender, el sketch despierta el driver y corre una prueba de motores (adelante, pausa, atrás) antes de empezar a seguir la línea.

```
/*
* ============================================================
* ROBOT: ARTURITO - SEGUIDOR DE LÍNEA AMATEUR (CON PRUEBA DE MOTORES)
* CONCURSO: 1er Concurso Distrital de Robótica 2026 (Santa Cruz)
* ============================================================
* Hardware:
* - Arduino UNO
* - 5 Sensores TCRT5000 (Analógicos en A0..A4)
* - Driver DRV8833 (Pines D5, D6, D9, D10)
* - 2 Motores HC02-48 (1:48)
*
* CALIBRACIÓN:
* Blanco: 30 - 40
* Negro:  970 - 990
* Umbral: 500 (Negro > 500)
*/

// --- DEFINICIÓN DE PINES ---
// Sensores (Entradas Analógicas)
#define S0 A0 // Extremo Izquierdo
#define S1 A1 // Centro Izquierdo
#define S2 A2 // Centro
#define S3 A3 // Centro Derecho
#define S4 A4 // Extremo Derecho
```

```
// Driver DRV8833 (Salidas PWM)
#define IN1 5  // Motor Izquierdo Adelante
#define IN2 6  // Motor Izquierdo Atrás
#define IN3 9  // Motor Derecho Adelante
#define IN4 10 // Motor Derecho Atrás
// EEP (sleep) del modulo a D4: HIGH = driver habilitado.
// Sin este cable el chip duerme y las salidas no conducen.
#define SLEEP_DRV 4

// --- PARÁMETROS DE VELOCIDAD Y CALIBRACIÓN ---
const int VEL_RAPIDA = 180;
const int VEL_MEDIA  = 150;
const int VEL_LENTA  = 120;

// Umbral ajustado según tus lecturas
const int UMBRAL = 500;

// Variable para recordar la última dirección conocida
int ultimoGiro = 0; // 0 = recto, 1 = izquierda, 2 = derecha
```

```
void setup() {
// Configurar pines del driver como salidas
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

// Despertar el DRV8833 (EEP en HIGH) antes de tocar motores
pinMode(SLEEP_DRV, OUTPUT);
digitalWrite(SLEEP_DRV, HIGH);

// Iniciar comunicación serial para depurar
Serial.begin(9600);

// Pausa de seguridad inicial (2 segundos)
delay(2000);

// ========================================
// PRUEBA DE MOTORES (Adelante y Retorno)
// ========================================
// 1. Avanzar un poco (800 milisegundos a velocidad media)
avanzar(VEL_MEDIA, VEL_MEDIA);
delay(800);
```

```
// 2. Detenerse brevemente
detener();
delay(300);

// 3. Retroceder exactamente el mismo tiempo para volver al punto inicial
retroceder(VEL_MEDIA, VEL_MEDIA);
delay(800);

// 4. Detenerse antes de arrancar con el seguidor de línea
detener();
delay(500);
}

void loop() {
// 1. LEER SENSORES (LOGICA INVERTIDA: > UMBRAL es Negro)
bool v0 = analogRead(S0) > UMBRAL;
bool v1 = analogRead(S1) > UMBRAL;
bool v2 = analogRead(S2) > UMBRAL;
bool v3 = analogRead(S3) > UMBRAL;
bool v4 = analogRead(S4) > UMBRAL;

// 2. LÓGICA DE MOVIMIENTO

// --- CASO 1: CENTRADO PERFECTO ---
```

```
if (v2 && !v0 && !v1 && !v3 && !v4) {
avanzar(VEL_RAPIDA, VEL_RAPIDA);
ultimoGiro = 0;
}
// --- CASO 2: DESVIADO LEVEMENTE A LA IZQUIERDA ---
else if (v3 || (v2 && v3)) {
avanzar(VEL_RAPIDA, VEL_LENTA);
ultimoGiro = 2;
}
// --- CASO 3: DESVIADO LEVEMENTE A LA DERECHA ---
else if (v1 || (v2 && v1)) {
avanzar(VEL_LENTA, VEL_RAPIDA);
ultimoGiro = 1;
}
// --- CASO 4: CURVA CERRADA A LA IZQUIERDA ---
else if (v4 || (v3 && v4)) {
girarDerecha();
ultimoGiro = 2;
}
// --- CASO 5: CURVA CERRADA A LA DERECHA ---
else if (v0 || (v0 && v1)) {
girarIzquierda();
ultimoGiro = 1;
}
// --- CASO 6: INICIO DE CURVA SUAVE ---
```

```
else if (v0 && !v1 && !v2) {
girarIzquierda();
ultimoGiro = 1;
}
else if (v4 && !v3 && !v2) {
girarDerecha();
ultimoGiro = 2;
}
// --- CASO 7: CRUCE ---
else if (v0 && v1 && v2 && v3 && v4) {
avanzar(VEL_RAPIDA, VEL_RAPIDA);
}
// --- CASO 8: LÍNEA PERDIDA ---
else if (!v0 && !v1 && !v2 && !v3 && !v4) {
if (ultimoGiro == 1) {
girarIzquierda();
} else if (ultimoGiro == 2) {
girarDerecha();
} else {
detener();
}
}
// --- CASO POR DEFECTO ---
```

```
else {
avanzar(VEL_MEDIA, VEL_MEDIA);
}

delay(5);
}

// ========================================
// FUNCIONES DE MOVIMIENTO
// ========================================

void avanzar(int velIzq, int velDer) {
analogWrite(IN1, velIzq);
digitalWrite(IN2, LOW);
analogWrite(IN3, velDer);
digitalWrite(IN4, LOW);
}

void retroceder(int velIzq, int velDer) {
digitalWrite(IN1, LOW);
analogWrite(IN2, velIzq);
digitalWrite(IN3, LOW);
analogWrite(IN4, velDer);
}
```

```
void girarIzquierda() {
analogWrite(IN1, 0);
digitalWrite(IN2, LOW);
analogWrite(IN3, VEL_RAPIDA);
digitalWrite(IN4, LOW);
}

void girarDerecha() {
analogWrite(IN1, VEL_RAPIDA);
digitalWrite(IN2, LOW);
analogWrite(IN3, 0);
digitalWrite(IN4, LOW);
}

void detener() {
digitalWrite(IN1, LOW);
digitalWrite(IN2, LOW);
digitalWrite(IN3, LOW);
digitalWrite(IN4, LOW);
}
```

Código — versión 2 (`firmware/arturito_v2/arturito_v2.ino`)

En vez de tabla de casos, calcula la posición de la línea como media ponderada de los cinco canales y corrige con un control proporcional-derivativo. La velocidad baja en las curvas, la calibración es automática (4 s al encender, pasando la línea por debajo) y, si pierde la línea, gira hacia el último lado que la vio y rebobina si no la recupera.

```
/*
* ============================================================
* ROBOT: ARTURITO v2 - SEGUIDOR DE LINEA CON CONTROL CONTINUO
* CONCURSO: 1er Concurso Distrital de Robotica 2026 (Santa Cruz)
* ============================================================
* Diferencias con v1:
* - Posicion continua (media ponderada de los 5 canales
*   analogicos) en vez de tabla de casos binaria.
* - Velocidad adaptativa: a fondo en rectas, baja en curvas.
* - Autocalibracion al encender: 4 s para pasar la linea por
*   los sensores; si no se calibra usa valores por defecto.
* - Recuperacion: gira hacia el ultimo lado que vio la linea y,
*   si no la encuentra, rebobina (retrocede buscando el punto).
* - Trim de motores: TRIM_IZQ/TRIM_DER corrigen la deriva en
*   recta (medir en el piso y ajustar a mano).
*
* Hardware: UNO + 5xTCRT5000 (A0..A4, flecha: S2 adelante,
* S1/S3 medios, S0/S4 atras en las puntas) + DRV8833 (IN1..IN4
* en D5/D6/D9/D10, EEP en D4, ULT en D2).
*/

// --- PINES ---
```

```
#define S0 A0 // extremo izquierdo (atras en la flecha)
#define S1 A1 // medio izquierdo
#define S2 A2 // centro (punta adelantada)
#define S3 A3 // medio derecho
#define S4 A4 // extremo derecho (atras en la flecha)

#define IN1 5   // motor izquierdo adelante (PWM)
#define IN2 6   // motor izquierdo atras (PWM)
#define IN3 9   // motor derecho adelante (PWM)
#define IN4 10  // motor derecho atras (PWM)
#define SLEEP_DRV 4 // EEP del DRV8833: HIGH = despierto
#define FAULT_PIN 2 // ULT del DRV8833 (opcional)

// --- PARAMETROS DE CONTROL ---
const int VEL_MAX = 200;   // velocidad en recta
const int VEL_MIN = 80;    // velocidad en curva cerrada
const float KP = 55.0;     // giro proporcional por unidad de posicion
const float KD = 18.0;     // amortigua el zigzag (derivada de la posicion)
const float KS = 60.0;     // cuanto baja la velocidad con el error

// Trim de motores (1.0 = igual). Si tira a la derecha, bajar TRIM_IZQ
// o subir TRIM_DER. Se ajusta a mano tras la prueba de recta.
const float TRIM_IZQ = 1.0;
const float TRIM_DER = 1.0;
```

```
// Recuperacion de linea perdida
const unsigned long GIRO_BUSQUEDA_MS = 1500; // tiempo girando antes de
rebobinar
const int VEL_BUSQUEDA = 130;
const int VEL_REBOBINADO = 90;

// Calibracion por defecto si la ventana no capto negro
const int CAL_MIN_DEF = 30;
const int CAL_MAX_DEF = 930;
const unsigned long CAL_MS = 4000;

// Posicion lateral de cada canal en la flecha: -2 izq .. +2 der
const float PESO[5] = {-2.0, -1.0, 0.0, 1.0, 2.0};
const int CANALES[5] = {S0, S1, S2, S3, S4};

int calMin[5];
int calMax[5];

float posPrev = 0;
int ultimoLado = 0;           // -1 izquierda, +1 derecha
unsigned long perdidaDesde = 0;

// --- MOTORES (velocidad con signo: + adelante, - atras) ---
```

```
void motorIzq(int vel) {
if (vel ⩾ 0) {
analogWrite(IN1, min(vel, 255));
digitalWrite(IN2, LOW);
} else {
digitalWrite(IN1, LOW);
analogWrite(IN2, min(-vel, 255));
}
}

void motorDer(int vel) {
if (vel ⩾ 0) {
analogWrite(IN3, min(vel, 255));
digitalWrite(IN4, LOW);
} else {
digitalWrite(IN3, LOW);
analogWrite(IN4, min(-vel, 255));
}
}

void detener() {
motorIzq(0); motorDer(0);
}

// --- CALIBRACION ---
```

```
void calibrar() {
Serial.println("CALIBRACION: pasa la linea por debajo de los");
Serial.println("sensores durante 4 s (blanco y negro).");
for (int i = 0; i < 5; i++) { calMin[i] = 1023; calMax[i] = 0; }

unsigned long t0 = millis();
while (millis() - t0 < CAL_MS) {
for (int i = 0; i < 5; i++) {
int v = analogRead(CANALES[i]);
if (v < calMin[i]) calMin[i] = v;
if (v > calMax[i]) calMax[i] = v;
}
delay(10);
}
```

```
// Si algun canal no vio contraste, usar valores por defecto
for (int i = 0; i < 5; i++) {
if (calMax[i] - calMin[i] < 150) {
calMin[i] = CAL_MIN_DEF;
calMax[i] = CAL_MAX_DEF;
}
Serial.print("S"); Serial.print(i);
Serial.print(": "); Serial.print(calMin[i]);
Serial.print("-"); Serial.println(calMax[i]);
}
}

// Negrura de un canal normalizada 0..1 segun su calibracion
float negrura(int i) {
int v = analogRead(CANALES[i]);
float n = (float)(v - calMin[i]) / (float)(calMax[i] - calMin[i]);
if (n < 0) n = 0;
if (n > 1) n = 1;
return n;
}
```

```
void setup() {
pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);
detener();

// Despertar el DRV8833 (EEP en HIGH)
pinMode(SLEEP_DRV, OUTPUT);
digitalWrite(SLEEP_DRV, HIGH);
pinMode(FAULT_PIN, INPUT_PULLUP);

Serial.begin(9600);
Serial.println("===== ARTURITO v2 =====");
calibrar();

// Prueba de motores igual que v1: adelante, pausa, atras
Serial.println("Prueba de motores...");
motorIzq(150); motorDer(150); delay(800);
detener(); delay(300);
motorIzq(-150); motorDer(-150); delay(800);
detener(); delay(500);
Serial.println("A seguir la linea.");
}
```

```
void loop() {
// 1. Lectura ponderada
float suma = 0, sumaP = 0;
bool todoNegro = true;
for (int i = 0; i < 5; i++) {
float n = negrura(i);
suma += n;
sumaP += PESO[i] * n;
if (n < 0.7) todoNegro = false;
}

// 2. Cruce o bifurcacion: todos negros → seguir recto a fondo
if (todoNegro) {
motorIzq(VEL_MAX); motorDer(VEL_MAX);
ultimoLado = 0;
perdidaDesde = 0;
delay(5);
return;
}

// 3. Linea a la vista: control proporcional-derivativo
if (suma > 0.3) {
float pos = sumaP / suma;               // -2 (izq) .. +2 (der)
float deriv = pos - posPrev;
posPrev = pos;
```

```
if (pos > 0.5) ultimoLado = 1;
else if (pos < -0.5) ultimoLado = -1;
perdidaDesde = 0;

int base = (int)(VEL_MAX - KS * abs(pos));
if (base < VEL_MIN) base = VEL_MIN;

float corr = KP * pos + KD * deriv;
int velIzq = (int)((base + corr) * TRIM_IZQ);
int velDer = (int)((base - corr) * TRIM_DER);
motorIzq(velIzq);
motorDer(velDer);
delay(5);
return;
}

// 4. Linea perdida: buscar hacia el ultimo lado visto
if (perdidaDesde == 0) perdidaDesde = millis();
unsigned long perdidaMs = millis() - perdidaDesde;
```

```
if (perdidaMs < GIRO_BUSQUEDA_MS) {
// Pivotar en el sitio hacia el ultimo lado conocido
int giro = (ultimoLado >= 0) ? VEL_BUSQUEDA : -VEL_BUSQUEDA;
motorIzq(giro);
motorDer(-giro);
} else {
// Rebobinar: retroceder despacio, todavia girando hacia ese lado
int giro = (ultimoLado >= 0) ? VEL_REBOBINADO : -VEL_REBOBINADO;
motorIzq(-VEL_REBOBINADO + giro / 2);
motorDer(-VEL_REBOBINADO - giro / 2);
}
delay(5);
}
```

Ambas versiones usan el mismo cableado. La versión 2 es la que corre en el robot; la 1 queda como referencia y respaldo.